Hybrid VQC Training on BCCC-CIC-IDS-2017 (Resumable)

This notebook trains a **Hybrid Variational Quantum Classifier (VQC)** on the BCCC-CIC-IDS-2017 dataset. It aligns with the updated QNN training workflow, including **checkpointing**, **advanced metrics**, and **resume capability**.

In [10]:
```
import pandas as pd
import numpy as np
import os
import torch
import torch.nn as nn
import pennylane as qml
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from torch.utils.data import DataLoader, TensorDataset
from tqdm import tqdm
import warnings
warnings.filterwarnings('ignore')
```

1. Configuration & Data Loading

In [16]:
```
# Configuration
DATASET_PATH = r'Datasets/processed_data.csv'
BATCH_SIZE = 32
EPOCHS = 15
LEARNING_RATE = 0.01
N_QUBITS = 4        # VQC Width
N_LAYERS = 2        # VQC Depth
SEED = 42

# Checkpoint Configuration
CHECKPOINT_DIR = 'checkpoints'
os.makedirs(CHECKPOINT_DIR, exist_ok=True)
CHECKPOINT_PATH = os.path.join(CHECKPOINT_DIR, 'checkpoint_ids2017_vqc.pth')

np.random.seed(SEED)
torch.manual_seed(SEED)
```

Out[16]:
```
<torch._C.Generator at 0x2320fd650d0>
```

In [17]:
```
print("Loading dataset...")
if not os.path.exists(DATASET_PATH):
    raise FileNotFoundError(f"Dataset not found at {DATASET_PATH}. Please run preprocess_da

df = pd.read_csv(DATASET_PATH)
print(f"Total samples: {len(df)}")

# Check for label column
if 'label' not in df.columns:
    raise ValueError("Column 'label' not found in dataset.")

# Separate Features and Target
X = df.drop(columns=['label']).values
y = df['label'].values

# Determine number of classes
num_classes = len(np.unique(y))
```

```python
print(f"Number of classes: {num_classes}")

# Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=SEED)

# Convert to PyTorch Tensors
X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train, dtype=torch.long)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test, dtype=torch.long)

# Adjust for Binary vs Multiclass
if num_classes == 2:
    y_train_tensor = y_train_tensor.float().unsqueeze(1)
    y_test_tensor = y_test_tensor.float().unsqueeze(1)
    criterion = nn.BCEWithLogitsLoss()
    output_dim = 1
    print("Binary Classification detected.")
else:
    criterion = nn.CrossEntropyLoss()
    output_dim = num_classes
    print("Multiclass Classification detected.")

# Create DataLoaders
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
test_dataset = TensorDataset(X_test_tensor, y_test_tensor)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

print(f"Training samples: {len(X_train)}")
print(f"Testing samples: {len(X_test)}")
print(f"Input Feature Dimension: {X_train.shape[1]}")
```

```
Loading dataset...
Total samples: 840549
Number of classes: 14
Multiclass Classification detected.
Training samples: 672439
Testing samples: 168110
Input Feature Dimension: 116
```

2. Hybrid VQC Architecture

In [18]:

```python
# Quantum Device
dev = qml.device("default.qubit", wires=N_QUBITS)

@qml.qnode(dev, interface="torch")
def vqc_circuit(inputs, weights):
    # Feature Map: Angle Embedding (more efficient for VQC)
    qml.AngleEmbedding(inputs, wires=range(N_QUBITS))

    # Ansatz: Strongly Entangling Layers
    qml.StronglyEntanglingLayers(weights, wires=range(N_QUBITS))

    # Measurement: Expectation values
    return [qml.expval(qml.PauliZ(i)) for i in range(N_QUBITS)]

class HybridVQC(nn.Module):
    def __init__(self, input_dim, n_qubits, n_layers, output_dim):
        super(HybridVQC, self).__init__()
        self.n_qubits = n_qubits

        # Classical Encoder: Project High-Dim -> Qubits
        self.cl_layer1 = nn.Linear(input_dim, 64)
        self.cl_layer2 = nn.Linear(64, n_qubits)
```

```
        # Quantum Layer (TorchLayer handles batching automatically)
        weight_shapes = {"weights": (n_layers, n_qubits, 3)}
        self.vqc = qml.qnn.TorchLayer(vqc_circuit, weight_shapes)

        # Classical Decoder: Qubits -> Classes
        self.cl_layer3 = nn.Linear(n_qubits, output_dim)

    def forward(self, x):
        # Classical encoding
        x = torch.relu(self.cl_layer1(x))
        x = self.cl_layer2(x)

        # Squash to [-pi, pi] for Angle Embedding
        x = torch.tanh(x) * np.pi

        # VQC
        x = self.vqc(x)

        # Final Classification
        x = self.cl_layer3(x)
        return x

model = HybridVQC(X_train.shape[1], N_QUBITS, N_LAYERS, output_dim)
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
print(model)
```

```
HybridVQC(
  (cl_layer1): Linear(in_features=116, out_features=64, bias=True)
  (cl_layer2): Linear(in_features=64, out_features=4, bias=True)
  (vqc): <Quantum Torch Layer: func=vqc_circuit>
  (cl_layer3): Linear(in_features=4, out_features=14, bias=True)
)
```

3. Training Loop with Resume Capability

In [19]:

```
def evaluate(model, loader, is_binary):
    model.eval()
    all_preds = []
    all_labels = []
    with torch.no_grad():
        for X_batch, y_batch in loader:
            logits = model(X_batch)
            if is_binary:
                preds = torch.sigmoid(logits) > 0.5
            else:
                preds = torch.argmax(logits, dim=1)

            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(y_batch.cpu().numpy())

    average_type = 'binary' if is_binary else 'weighted'
    acc = accuracy_score(all_labels, all_preds)
    prec = precision_score(all_labels, all_preds, average=average_type, zero_division=0)
    rec = recall_score(all_labels, all_preds, average=average_type, zero_division=0)
    f1 = f1_score(all_labels, all_preds, average=average_type, zero_division=0)
    return acc, prec, rec, f1

# Check for checkpoint
start_epoch = 0
if os.path.exists(CHECKPOINT_PATH):
    print(f"Loading checkpoint from {CHECKPOINT_PATH}...")
    checkpoint = torch.load(CHECKPOINT_PATH)
    model.load_state_dict(checkpoint['model_state_dict'])
    optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
    start_epoch = checkpoint['epoch'] + 1
```

```python
    print(f"Resuming training from epoch {start_epoch+1}")
else:
    print("No checkpoint found. Starting training from scratch.")

print("Starting training...")
is_binary = (num_classes == 2)

if start_epoch < EPOCHS:
    for ep in range(start_epoch, EPOCHS):
        model.train()
        total_loss = 0
        for X_batch, y_batch in tqdm(train_loader, desc=f"Epoch {ep+1}/{EPOCHS}"):
            optimizer.zero_grad()
            logits = model(X_batch)
            loss = criterion(logits, y_batch)
            loss.backward()
            optimizer.step()
            total_loss += loss.item()

        avg_loss = total_loss / len(train_loader)
        acc, prec, rec, f1 = evaluate(model, test_loader, is_binary)

        print(f"Epoch {ep+1}: Loss = {avg_loss:.4f}")
        print(f"  Test Accuracy: {acc:.4f}, Precision: {prec:.4f}, Recall: {rec:.4f}, F1: {

        # Save Checkpoint
        checkpoint = {
            'epoch': ep,
            'model_state_dict': model.state_dict(),
            'optimizer_state_dict': optimizer.state_dict(),
            'loss': avg_loss,
        }
        torch.save(checkpoint, CHECKPOINT_PATH)
        print(f"Checkpoint saved for Epoch {ep+1}")
else:
    print("Training already completed!")

# Save Final Model
save_dir = 'trained_model'
os.makedirs(save_dir, exist_ok=True)
model_path = os.path.join(save_dir, 'hybrid_vqc_ids2017.pth')
torch.save(model.state_dict(), model_path)
print(f"Final Model saved to {model_path}")
```

```
No checkpoint found. Starting training from scratch.
Starting training...
Epoch 1/15: 100%|██████████| 21014/21014 [09:27<00:00, 37.04it/s]
Epoch 1: Loss = 0.2601
  Test Accuracy: 0.9171, Precision: 0.9048, Recall: 0.9171, F1: 0.9093
Checkpoint saved for Epoch 1
Epoch 2/15: 100%|██████████| 21014/21014 [2:17:09<00:00,  2.55it/s]
Epoch 2: Loss = 0.2287
  Test Accuracy: 0.9127, Precision: 0.9037, Recall: 0.9127, F1: 0.9013
Checkpoint saved for Epoch 2
Epoch 3/15: 100%|██████████| 21014/21014 [10:40<00:00, 32.78it/s]
Epoch 3: Loss = 0.3162
  Test Accuracy: 0.9100, Precision: 0.9009, Recall: 0.9100, F1: 0.9025
Checkpoint saved for Epoch 3
Epoch 4/15: 100%|██████████| 21014/21014 [09:08<00:00, 38.30it/s]
Epoch 4: Loss = 0.3218
  Test Accuracy: 0.9147, Precision: 0.9124, Recall: 0.9147, F1: 0.9085
Checkpoint saved for Epoch 4
Epoch 5/15: 100%|██████████| 21014/21014 [09:09<00:00, 38.25it/s]
Epoch 5: Loss = 0.3129
  Test Accuracy: 0.9185, Precision: 0.9115, Recall: 0.9185, F1: 0.9110
Checkpoint saved for Epoch 5
```

```
Epoch 6/15: 100%|██████████| 21014/21014 [09:09<00:00, 38.26it/s]
Epoch 6: Loss = 0.3379
  Test Accuracy: 0.9025, Precision: 0.8898, Recall: 0.9025, F1: 0.8935
Checkpoint saved for Epoch 6
Epoch 7/15: 100%|██████████| 21014/21014 [12:55<00:00, 27.10it/s]
Epoch 7: Loss = 0.3127
  Test Accuracy: 0.9139, Precision: 0.8938, Recall: 0.9139, F1: 0.9010
Checkpoint saved for Epoch 7
Epoch 8/15: 100%|██████████| 21014/21014 [11:34<00:00, 30.26it/s]
Epoch 8: Loss = 0.3196
  Test Accuracy: 0.8939, Precision: 0.8762, Recall: 0.8939, F1: 0.8775
Checkpoint saved for Epoch 8
Epoch 9/15: 100%|██████████| 21014/21014 [09:57<00:00, 35.15it/s]
Epoch 9: Loss = 0.3676
  Test Accuracy: 0.9010, Precision: 0.8819, Recall: 0.9010, F1: 0.8854
Checkpoint saved for Epoch 9
Epoch 10/15: 100%|██████████| 21014/21014 [11:10<00:00, 31.33it/s]
Epoch 10: Loss = 0.3632
  Test Accuracy: 0.9044, Precision: 0.8942, Recall: 0.9044, F1: 0.8902
Checkpoint saved for Epoch 10
Epoch 11/15: 100%|██████████| 21014/21014 [11:27<00:00, 30.55it/s]
Epoch 11: Loss = 0.3372
  Test Accuracy: 0.9027, Precision: 0.8896, Recall: 0.9027, F1: 0.8903
Checkpoint saved for Epoch 11
Epoch 12/15: 100%|██████████| 21014/21014 [10:23<00:00, 33.69it/s]
Epoch 12: Loss = 0.3412
  Test Accuracy: 0.9036, Precision: 0.8899, Recall: 0.9036, F1: 0.8913
Checkpoint saved for Epoch 12
Epoch 13/15: 100%|██████████| 21014/21014 [10:32<00:00, 33.25it/s]
Epoch 13: Loss = 0.3458
  Test Accuracy: 0.9058, Precision: 0.9016, Recall: 0.9058, F1: 0.8937
Checkpoint saved for Epoch 13
Epoch 14/15: 100%|██████████| 21014/21014 [11:54<00:00, 29.42it/s]
Epoch 14: Loss = 0.3359
  Test Accuracy: 0.9035, Precision: 0.8922, Recall: 0.9035, F1: 0.8907
Checkpoint saved for Epoch 14
Epoch 15/15: 100%|██████████| 21014/21014 [09:53<00:00, 35.41it/s]
Epoch 15: Loss = 0.3357
  Test Accuracy: 0.9074, Precision: 0.8959, Recall: 0.9074, F1: 0.8930
Checkpoint saved for Epoch 15
Final Model saved to trained_model\hybrid_vqc_ids2017.pth
```